Hinweis zum Urheberrecht

Copyright Notice

Robotics, Artificial Intelligence
and Real-time Systems

Reinforcement Learning:
Monte Carlo Methods

Prof. Dr.-Ing. habil. Alois Knoll
Kejia Chen, M.Sc.

MC Prediction

- In practical cases, the MDP is usually unknown ($< \mathcal{S}, \mathcal{A}, \cancel{\mathcal{P}}, \cancel{\mathcal{R}}, \gamma >$).
- Monte-Carlo-based approaches don't require knowledge of the MDP as they learn from sampled state trajectories:

- **Monte Carlo prediction:** Return is calculated for all states in each sampled trajectory. For the evaluation of value functions, experienced returns are averaged.

π	policy	R	stochastical reward function
A	stochastical action function	S	stochastical state function
q	value for a specific action	T	terminal state

[§]*Sutton, R. S. and Barto, A. G. Reinforcement learning: An introduction. 2nd ed. Adaptive computation and machine learning. Cambridge, Massachusetts: The MIT Press, 2018,* ***p.92.***
Slide is adapted from CogSys21 RL lecture held by Prof. Dr. Ing. Noah Klarmann.

MC Prediction: Pseudo Code

Input: a policy π to be evaluated
Initialize:
    $V(s) \in \mathbb{R}$, arbitrarily, for all $s \in \mathcal{S}$
    $Returns(s) \leftarrow$ an empty list, for all $s \in \mathcal{S}$
Loop forever (for each episode):
    Generate an episode following $\pi : S_0, A_0, R_1, S_1, A_1, R_2, \ldots, S_{T-1}, A_{T-1}, R_T$
    $G \leftarrow 0$
    Loop for each step of episode, $t = T-1, T-2, \ldots, 0$:
        $G \leftarrow \gamma G + R_{t+1}$
        Unless S_t appears in $S_0, S_1, \ldots, S_{t-1}$: — first visit to S_t
            Append G to $Returns(S_t)$
            $V(S_t) \leftarrow \text{average}(Returns(S_t))$

Incremental implementation:
$V(S_t) \leftarrow V(S_t) + \alpha(G_t - V(S_t))$

γ	discount rate
π	policy
A	stochastical action function
G	return
q	value for a specific action
R	stochastical reward function
S	stochastical state function
s	particular state
V	value estimates
α	learning rate

[§]Sutton, R. S. and Barto, A. G. Reinforcement learning: An introduction. 2nd ed. Adaptive computation and machine learning. Cambridge, Massachusetts: The MIT Press, 2018, ***p.92.***
Slide is adapted from CogSys21 RL lecture held by Prof. Dr. Ing. Noah Klarmann

MC Control

- Without model, state values alone are not sufficient to determine a policy. One of primary goals for Monte Carlo methods is to estimate **state-action values**.
- Exploring starts: If π is a deterministic policy, many state-action pairs may never be visited.

- **Monte Carlo control:** updating value and policy for every sample:

$$\pi_0 \xrightarrow{eval.} q_{\pi_0} \xrightarrow{impr.} \pi_1 \xrightarrow{eval.} q_{\pi_1} \xrightarrow{impr.} \pi_2 \dots \pi_* \xrightarrow{eval.} q_*$$

π	policy	R	stochastical reward function
A	stochastical action function	S	stochastical state function
q	value for a specific action	T	terminal state

[§]Sutton, R. S. and Barto, A. G. Reinforcement learning: An introduction. 2nd ed. Adaptive computation and machine learning. Cambridge, Massachusetts: The MIT Press, 2018, ***p.92,97.*** *Slide is adapted from CogSys21 RL lecture held by Prof. Dr. Ing. Noah Klarmann*

MC Control: With Exploring Starts

Initialize:
  $\pi(s) \in \mathcal{A}(s)$ (arbitrarily), for all $s \in \mathcal{S}$
  $Q(s, a) \in \mathbb{R}$ (arbitrarily), for all $s \in \mathcal{S}, a \in \mathcal{A}(s)$
  $Returns(s, a) \leftarrow$ empty list, for all $s \in \mathcal{S}, a \in \mathcal{A}(s)$

Loop forever (for each episode):
  Choose $S_0 \in \mathcal{S}, A_0 \in \mathcal{A}(S_0)$ randomly such that all pairs have probability > 0 — Exploring starts
  Generate an episode from S_0, A_0, following $\pi : S_0, A_0, R_1, \ldots, S_{T-1}, A_{T-1}, R_T$
  $G \leftarrow 0$
  Loop for each step of episode, $t = T-1, T-2, \ldots, 0$:
    $G \leftarrow \gamma G + R_{t+1}$
    Unless the pair S_t, A_t Appears in $S_0, A_0, S_1, A_1, \ldots, S_{t-1}, A_{t-1}$:
      Append G to $Returns(S_t, A_t)$
      $Q(S_t, A_t) \leftarrow \text{average}(Returns(S_t, A_t))$ — Estimate state-action values
      $\pi(S_t) \leftarrow \text{argmax}_a Q(S_t, a)$ — greedy

ϵ	probability of random action
γ	discount rate
π	policy
A	stochastical action function
G	return
q	value for a specific action
R	stochastical reward function
S	stochastical state function
s	particular state
V	value estimates

[§]*Sutton, R. S. and Barto, A. G. Reinforcement learning: An introduction. 2nd ed. Adaptive computation and machine learning. Cambridge, Massachusetts: The MIT Press, 2018,* ***p.99.***

MC Control: Without Exploring Starts

Algorithm parameter: small $\epsilon > 0$
Initialize:
  $\pi \leftarrow$ an arbitrary ϵ-soft policy
  $Q(s,a) \in \mathbb{R}$ (arbitrarily), for all $s \in \mathcal{S}, a \in \mathcal{A}(s)$
  $Returns(s,a) \leftarrow$ an empty list, for all $s \in \mathcal{S}, a \in \mathcal{A}(s)$

Repeat forever (for each episode):
  Generate an episode following $\pi : S_0, A_0, R_1, S_1, A_1, R_2, ..., S_{T-1}, A_{T-1}, R_T$
  $G \leftarrow 0$
  Loop for each step of episode, $t = T-1, T-2, ..., 0$:
    $G \leftarrow \gamma G + R_{t+1}$
    Unless the pair S_t, A_t appears in $S_0, A_0, S_1, A_1, ..., S_{t-1}, A_{t-1}$:
      Append G to $Returns(S_t, A_t)$
      $Q(S_t, A_t) \leftarrow \text{average}(Returns(S_t, A_t))$
      $A* \leftarrow \arg\max_a Q(S_t, a)$ (with ties broken arbitrarily)
      For all $a \in \mathcal{A}(S_t)$:

$$\pi(a|S_t) \leftarrow \begin{cases} 1 - \epsilon + \epsilon/|\mathcal{A}(S_t)| & \text{if } a = A* \\ \epsilon/|\mathcal{A}(S_t)| & \text{if } a \neq A* \end{cases}$$

ϵ	probability of random action
γ	discount rate
π	policy
A	stochastical action function
G	return
q	value for a specific action
R	stochastical reward function
S	stochastical state function
s	particular state
V	value estimates

ϵ −greedy: Keeping a non-zero probability for every action. This ensures an exhaustive exploration of all states to guarantee convergence towards an optimal policy.

[§]*Sutton, R. S. and Barto, A. G. Reinforcement learning: An introduction. 2nd ed. Adaptive computation and machine learning. Cambridge, Massachusetts: The MIT Press, 2018,* ***p.101.***
Slide is adapted from CogSys21 RL lecture held by Prof. Dr. Ing. Noah Klarmann